



ML 2025 Project

Matteo Cherubini ~ m.cherubini6@studenti.unipi.it

Computer Science, Artificial Intelligence

Team: gwynn7

Date: 20/08/2026

Type: **A (Neural Network C++)**

Objective & Implementation

Objective

- Develop a customizable neural network simulator written in **C++23**, to learn how to implement various machine learning techniques generalizable on different tasks and datasets

Aims

- Test the framework on the three **MONK** [10] problems, with particular focus on the effect of regularization on **MONK 3**
- Perform a dedicated model selection to find the best configuration for **ML-CUP** dataset
- Visualize how the implemented features influence model performance on hyperparameters or training context change

Framework:

- **C++23**
- **Eigen** [7] library for Matrix Computation
- **CLI11** [6] library for CLI argument parsing
- *OpenBLAS* [9] & *OpenMP* [8] for efficiency and multithreaded processing (optional via CMake)
- UNIX-only signal handling and training launch wrapper

Implementation:

- **Scalar** compile-time definition (single or double precision, via CMake - **used *float***)
- **OOP** for Network, Layers, Optimizers and Normalizers
- CSV files for dynamic and iterable *model structure* definition (grid search)
- *Dumping* and *Loading* of trained models (.bin) ²

Machine Learning Framework

Neural Network Structure

- Modular stacking of Layers:
 - **Dense, Activation** and Dropout
- Activation Functions:
 - Linear, **Sigmoid, Tanh, ReLU** [5], Leaky ReLU and Softmax
- Loss Functions:
 - **MSE, MEE, BCE, CCE**
- Initialization Methods:
 - Random, **LeCun, Glorot** [3], **He** [4]
- Optimizer:
 - **SGD (Momentum), RMSProp, Adam** (*Decoupled Weight Decay*) [1,2]
- Regularization schema:
 - **L2**, Decoupled Weight Decay, Dropout

Training Framework

- Methods:
 - **Online, Batch** and **Minibatch**
- Model Selection Criteria:
 - **MEE, MSE**, Error Rate (*with Custom Metric*)
- Learning Rate Schedule:
 - Warmup, **Linear Decay**
- Early Stopping Rules:
 - Fixed, **Patience, Error Level**
- Normalization Methods:
 - None, **Min-Max**, Abs-Max, **Z-Score**
- Datasets implemented:
 - XOR, **MONK** [10], **ML-CUP**, MNIST [11]
- Grid Search:
 - Holdout, Standard or Stratified K-Fold, **Nested K-Fold**

Beyond the Lectures

Use Updates Count instead of Epochs (derived at runtime)

- This grants fairness between **mini-batch** and **full-batch** trainings with respect to optimization steps.
- This also allows the correct functioning of **warmup** and **linear decay** without the need of constant re-tuning.

Model Selection Metric: Select smooth and stable models

- Model Score = Minimum of the scores over a *sliding window*.
- **Window Score = Window Mean + 0.5 * Window Std**

Reproducibility

- The starting **seed** (used **42**) is combined with the **indices** of the Trial, the Inner Fold and the Outer Fold.
- This allows for new random initializations and batch shuffling for each trial, but reproducible from the seed.

Optimizers: *RMS Prop*, *Adam* [1]

Learning Rate Rules: *Warm Up*, *Linear Decay*

Weight Initialization: *LeCun*, *Glorot* [3], *He* [4]

Data Normalization: *Min-Max*, *Abs-Max*, *Z-Score*

K-Fold CV: *Stratified*, *Nested*

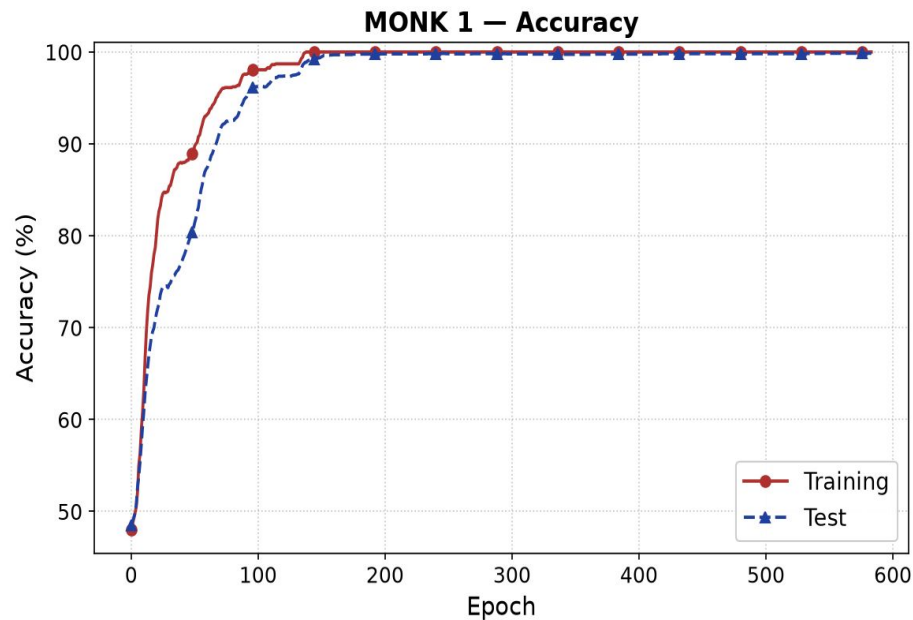
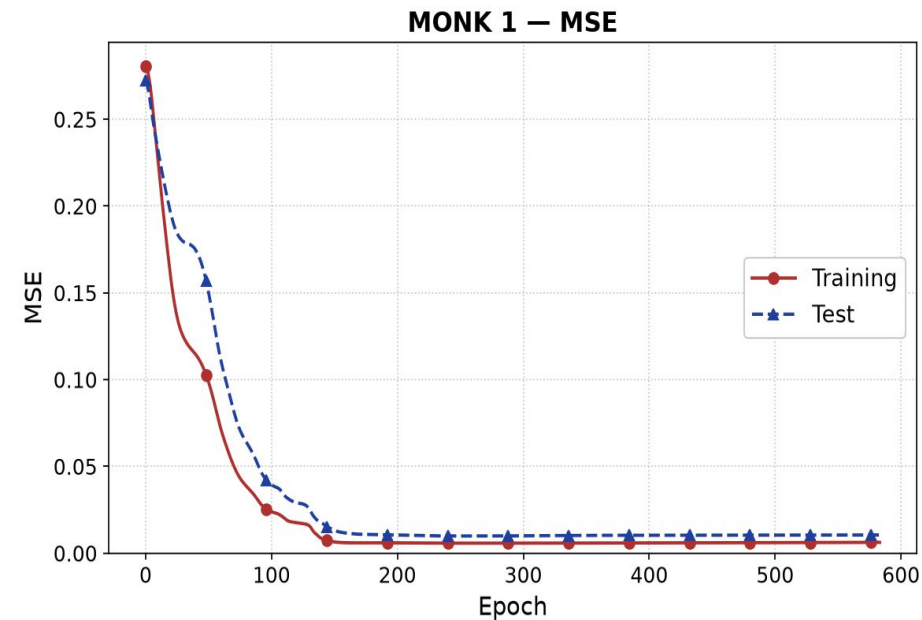
Multiclass Classification: *Softmax + Categorical Cross-Entropy*

Dataset: *MNIST* [11]

Monk Results: Table

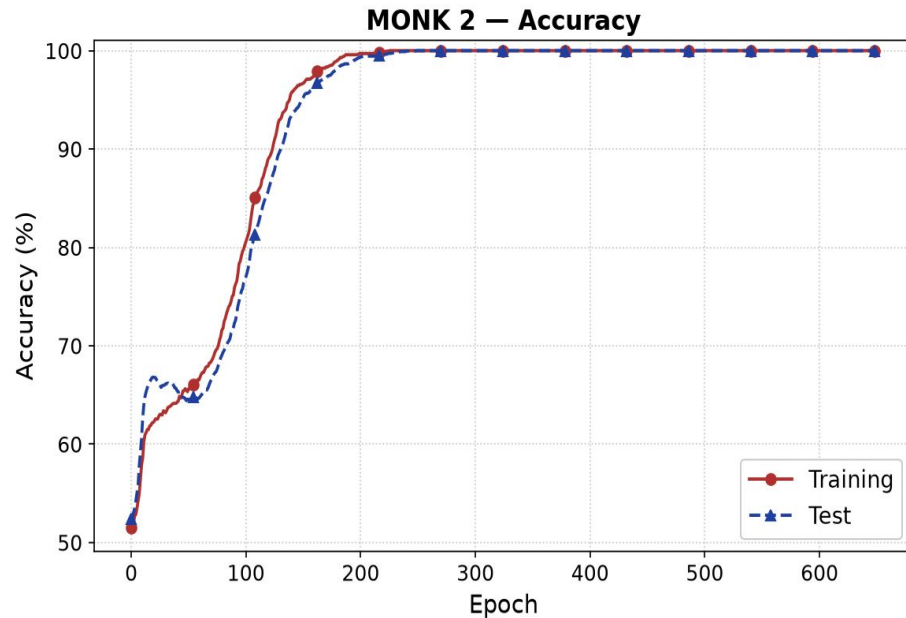
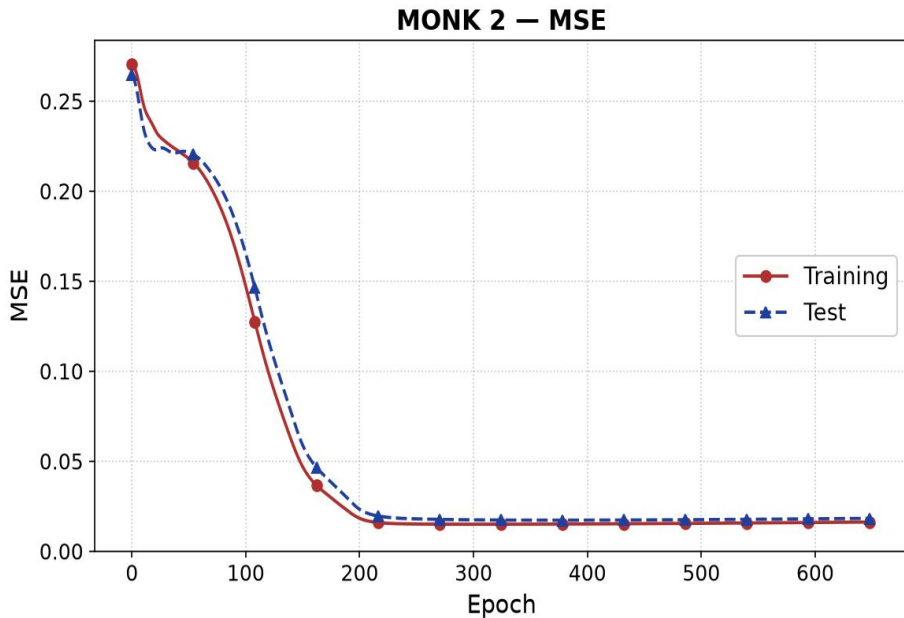
Problem	Units, η , λ , α	MSE (TR/TS)	Accuracy (TR/TS)
MONK 1	4, 2, 0.01, 0.85	0.005 / 0.009	100.00% / 99.77%
MONK 2	4, 0.5, 0.01, 0.9	0.014 / 0.016	100.00% / 100.00%
MONK 3 (reg)	3, 0.5, 0.01, 0.85	0.046 / 0.037	94.18% / 96.81%
MONK 3 (no reg)	3, 0.5, 0, 0.85	0.012 / 0.056	98.77% / 93.33%

Monk 1 Results: Plot



Hidden: Tanh - Output: Sigmoid - Init: Glorot - Loss: MSE - Optimizer: SGD - Normalization: None

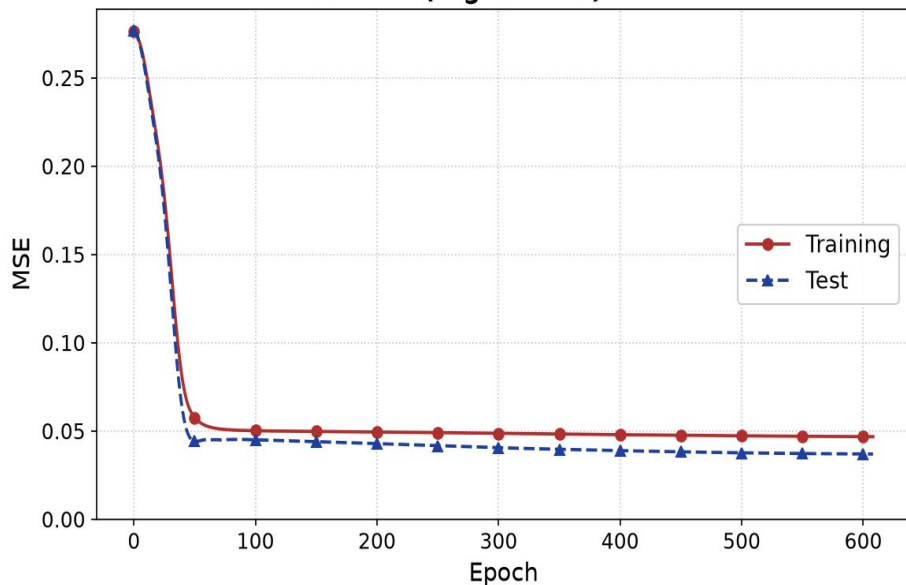
Monk 2 Results: Plot



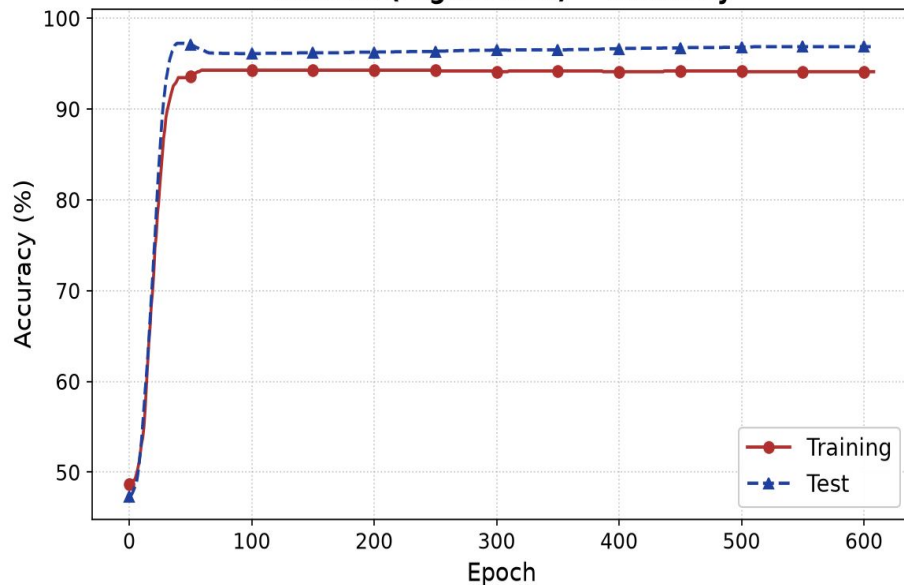
Hidden: Tanh - Output: Sigmoid - Init: Glorot - Loss: MSE - Optimizer: SGD - Normalization: None

Monk 3 Results: Plot - L2 Regularization

MONK 3 (regularized) — MSE



MONK 3 (regularized) — Accuracy



Hidden: Tanh - Output: Sigmoid - Init: Glorot - Loss: MSE - Optimizer: SGD - Normalization: None

CUP Validation: Model Selection & Data Splitting

Nested CV Methodology:

- Models selected via **Inner Cross Validation**, calculating the score of each model on the **Validation Set**
- For each **Inner Fold**, multiple **Trials** are performed, and the final results are averaged to get one **Run Score**
- The selected models for each **Outer Fold** are re-trained on the full **Train+Valid Set**
- If **Outer Folds** disagree on the best model, the most frequently chosen one is selected (or the *least complex*)
- Final evaluation of the split is performed on the **Internal Test Set** (Outer Folds' Holdout)
- The *final model* is re-trained on the entire dataset and saved to disk to generate the blind predictions

Search Phase	Trials	Inner CV	Outer CV	Purpose
Screening	2	4-Fold	Holdout 80/20	Establishing general trends and promising models
Tuning	5	5-Fold	Holdout 80/20	Precise tuning of numeric hyperparameters
Refinement	15	5-Fold	5-Fold	Comparing the best architectures across every outer fold
Assessment	15	5-Fold	5-Fold	Selecting the final model for evaluation and predictions

CUP Validation: Grid Search & Hyperparameters

Hyperparameter Category	Explored Values & Ranges
Network Architecture	1 to 6 layers, 8 to 384 units
Activation Functions	Tanh, Sigmoid, ReLU
Weight Initialization	Glorot, He, LeCun
Normalization	Z-Score, MinMax
Optimizer	Adam, SGD, RMSProp
Learning Rate	5e-4 to 1e-2
L2 Regularization	5e-4 to 1e-1
Batch Size	16, 32, 64, 128, Full Batch
Epochs / Early Stopping	Max 25000 Updates / Patience, Error Level
Other	Loss (MSE, MEE), Dropout

Screening Details

Isolate core hyperparameters early to
avoid combinatorial search space

Executed in Linux environment

CPU: Ryzen 5 5600X | RAM: 16GB DDR4
[OpenBlas & OpenMP Multithreading]

Optimizer: *Adam* outperformed *RMSProp* and *SGD* across all topologies, especially in deep networks.

Initialization: The optimal *weights initialization* methods were bounded to the choice of the activation function (*Tanh+Glorot*, *ReLU+He*).

Capacity: Small networks can learn but *underfit*, a minimum of 3 layers and ~64 units is required.

Interactions: Shallow networks work better with *Tanh+Z-Score*, while Deep networks work better with *ReLU+Min-Max*.

Batch Size: For a max of 500 samples, *Full Batch* training eliminates *Mini-Batch* noise and generally performs better.

Convergence: *Patience* early stopping allowed for faster training with no loss in performance.

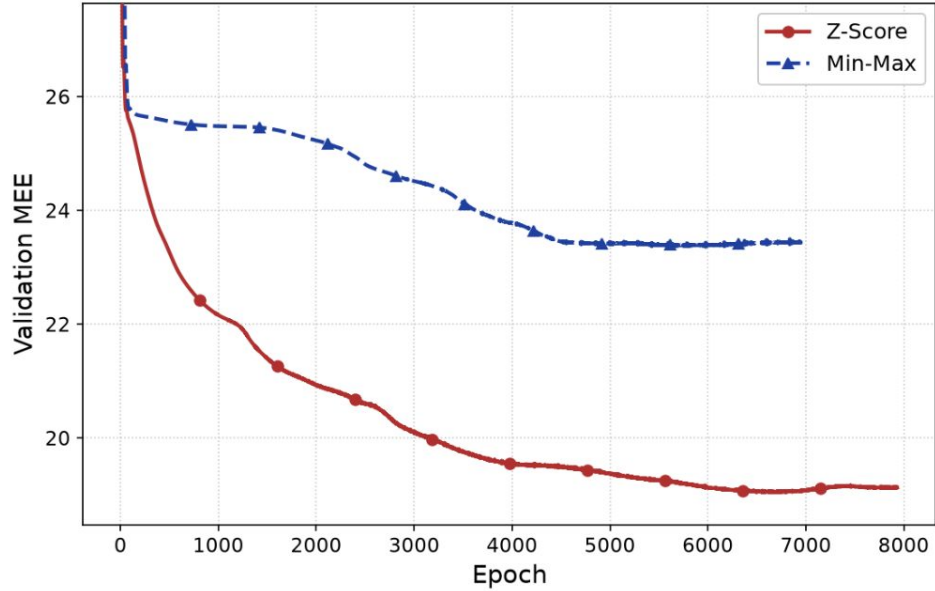
CUP Results: Refinement Validation Results

Type	Network	Hidden	Init	Optim	Norm	Batch	η	λ	$\beta 1$	$\beta 2$
Wide	160x4	ReLU	He	Adam	MinMax	Full	0.0009	0.001	0.9	0.999
Deep	128x5	ReLU	LeCun	Adam	MinMax	Full	0.001	0.001	0.9	0.999
Shallow	96-64-32	Tanh	Glorot	Adam	ZScore	Full	0.002	0.0025	0.9	0.999

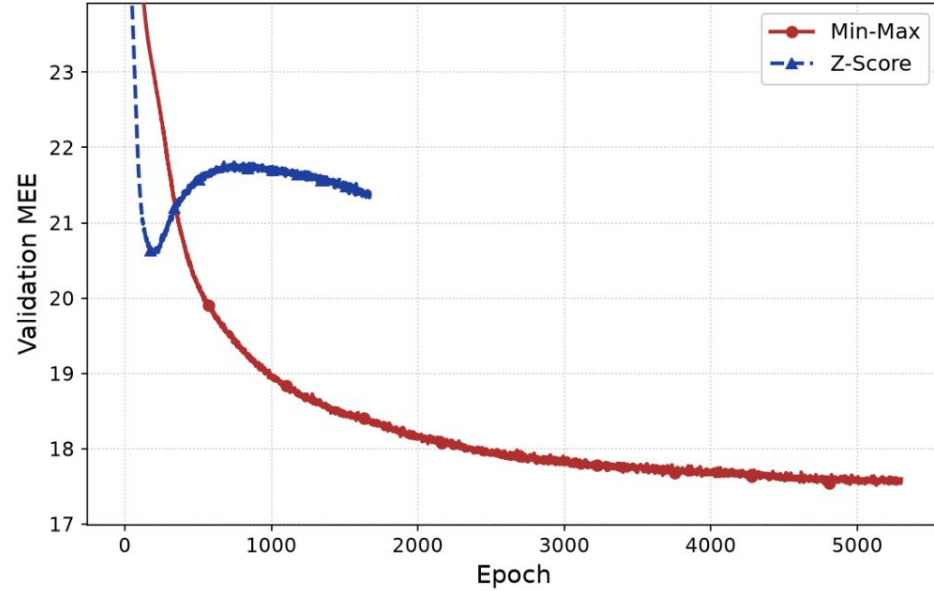
Type	Train MEE	Test MEE	Valid MEE	Valid $\pm$ Runs	Run Time	Total Time	Epoch Time
Wide	8.63 $\pm$ 0.14	17.44 $\pm$ 0.58	17.59 $\pm$ 0.24	$\pm$ 1.18	5.9s	7m 21s	1.28ms
Deep	9.97 $\pm$ 0.16	17.14 $\pm$ 0.43	17.51 $\pm$ 0.14	$\pm$ 1.13	7.0s	8m 42s	1.31ms
Shallow	11.59 $\pm$ 0.35	16.77 $\pm$ 0.69	17.47 $\pm$ 0.31	$\pm$ 1.30	2.7s	3m 19s	0.36ms

Train, Test and Valid MEE are averaged within each outer fold, so $\pm$ is across the 5 outer folds. Valid $\pm$ Runs instead measures variability across all runs (5 outer $\times$ 5 inner $\times$ 15 trials)

Shallow Tanh 96-48-24

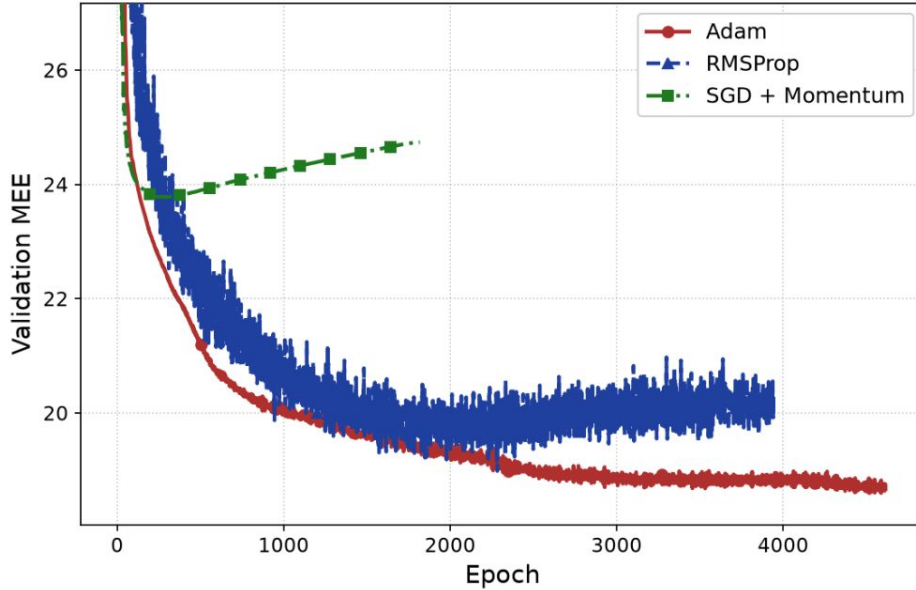


Deep ReLU 128x5

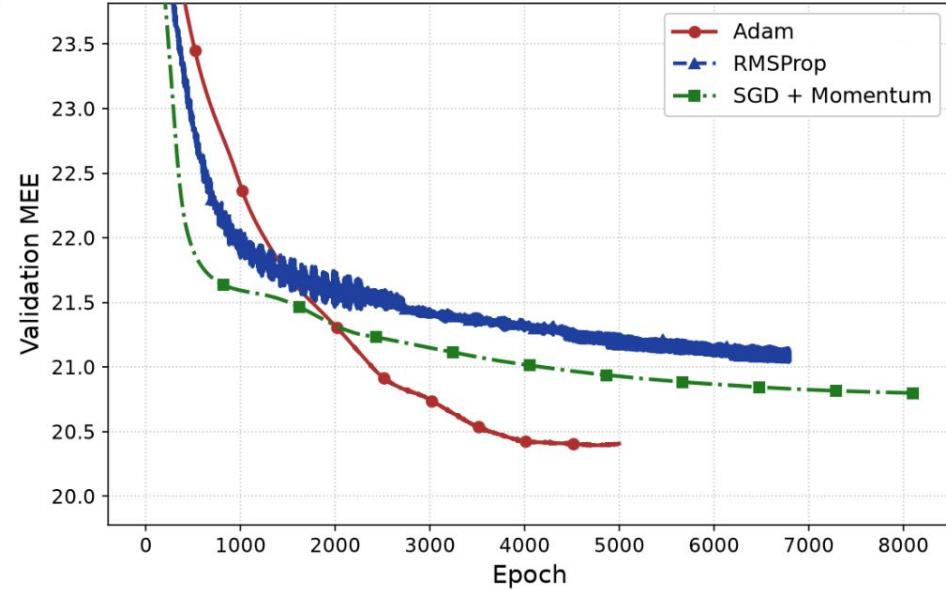


Impact of normalization methods on different architectures, showing the clear superiority of each combination

Deep ReLU 128x4 (Min-Max)

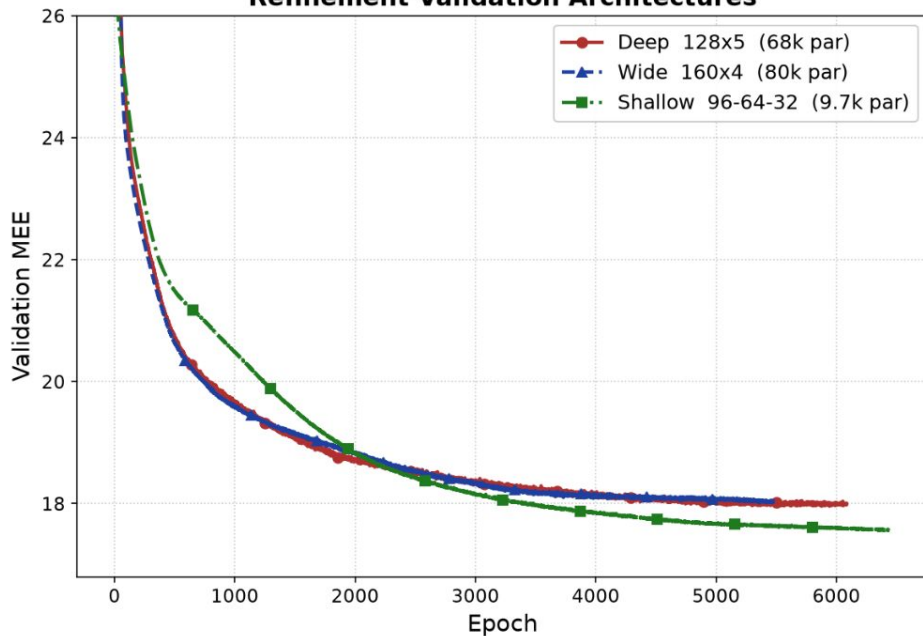


Shallow Tanh 64-32 (Z-Score)

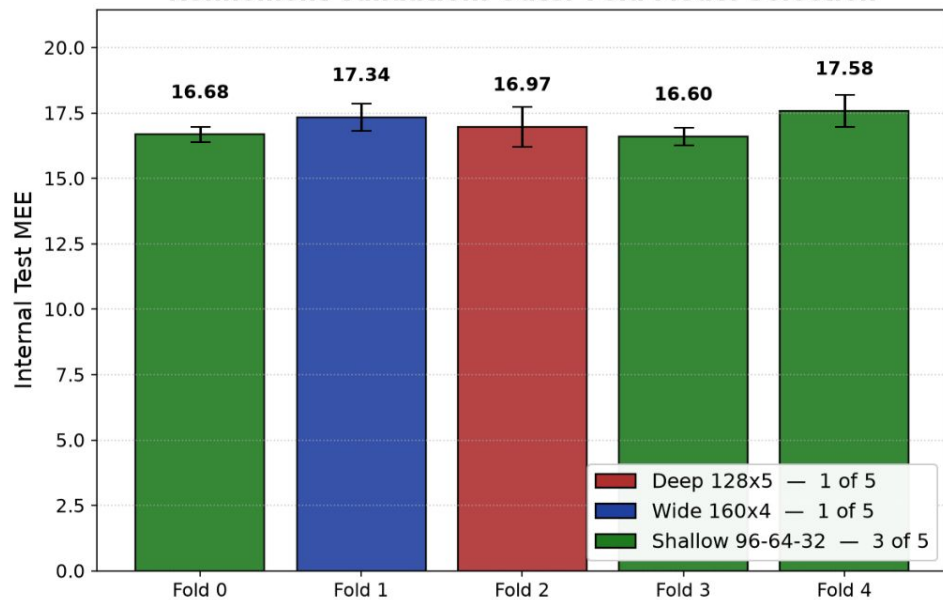


Convergence behaviour of models trained with different optimizers, showing Adam superiority in Screening

Refinement Validation Architectures



Refinement Validation: Outer Fold Model Selection



Validation Learning Curve and Outer Selection of the best architectures in Refinement Schema [15 Trials]

CUP Results: Choosing Final Model

Previous plots, and graphs in the Appendix,
bring proof for some of these claims

Noise and Smoothness:

- Noise decreases with *full batches*, and increases with *deep models* if not properly tuned
- **Choice:** Full Batch

Architecture

- *Optimizer* and *Normalization+Hidden Activation* patterns were evident and consistent
- **Choice:** *Adam*, *Tanh+ZScore* or *ReLU+MinMax*

Complexity

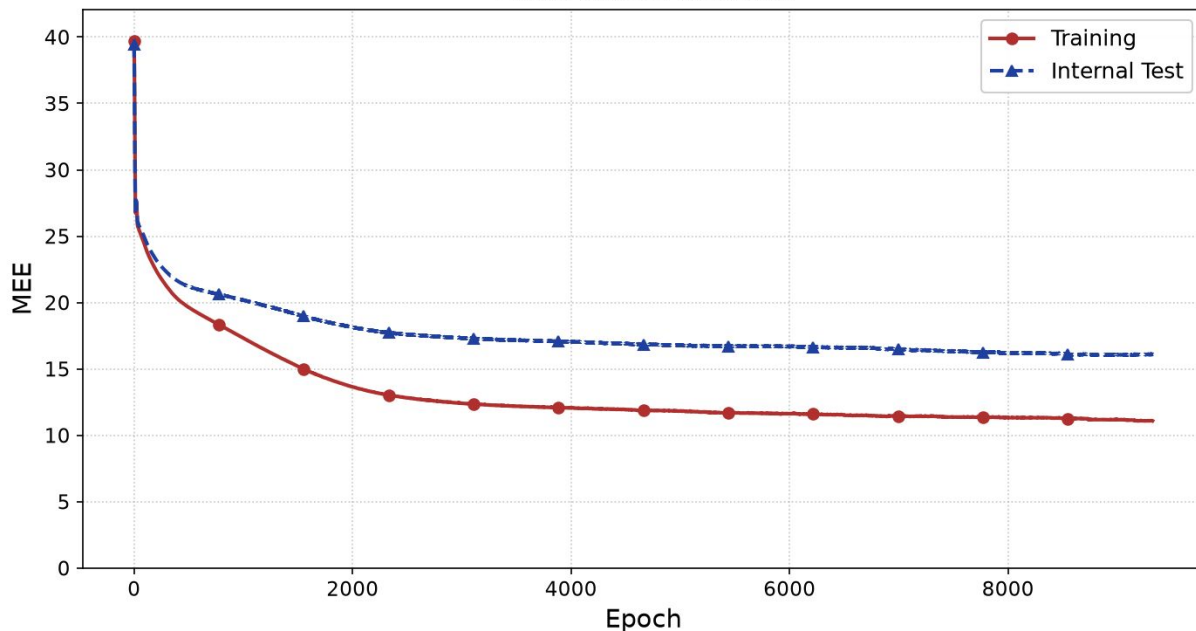
- Shallow models, even though harder to tune, were able to compete with deeper models with a fraction of the parameters
- The shallow model, other than being the least complex, was also the most frequently selected one by the custom metric
- **Choice:** *Shallow Model*

Parameters

- **Deep Model:** ~68k parameters (128x5)
- **Wide Model:** ~80k parameters (160x4)
- **Shallow Model:** ~10k parameters (96-64-32)

CUP Results: Final Model

Final Model Retrain



Configuration & Metrics	Value
Architecture	96-64-32 (Shallow)
Hidden Activation	Tanh
Weight Initialization	Glorot
Optimizer	Adam
Data Normalization	Z-Score
Batch Size	Full Batch
Learning Rate (η)	0.002
L2 Regularization (λ)	0.0025
Adam Moments (β_1, β_2)	0.9, 0.999
Trainable Parameters	9,668
Train MEE	11.59 ± 0.35
Validation MEE	17.47 ± 0.31
Test MEE	16.77 ± 0.69
Epoch Time	~0.36 ms
Per Run Retrain Time	~2.7 s

Validation MEE comes from the inner folds of the Refinement run. Train and Test come from the outer folds of the Assessment run. All $\pm$ are across the 5 outer folds

Discussion: Novelties

Visual proofs available as graphs in the Appendix

Update-based Pipeline: Calculating *Epochs* dynamically prevented *Warmup*, *Decay*, and *Patience* from altering meaning when *batch sizes* changed and ensured a fair comparisons across early tests, but mini-batch training was discarded in the screening phase anyways.

Model Selection Metric: Didn't help finding a model with drastically lower *best MEE*, but it proved essential to select the most stable and reliable models by focusing more often on the same choice across multiple runs, both in early screening and in final selections (where the standard metric was close to a tie, while the custom one concentrated on the ***Shallow model***).

Optimizer Choice: On ***Shallow models***, ***SGD*** remains a valid alternative, even though slower and generally worse. On deeper networks, however, ***Adam*** proved to be essential rather than only slightly better.

Normalization Synergy: Normalization method proved to be strictly bound to both *network complexity* and *hidden activation* function, with ***Z-Score+Tanh Shallow*** competing with ***Min-Max+ReLU Deep*** models.

Interaction Grid Search: Avoided a fully combined grid by analyzing interacting hyperparameters to prevent combinatorial search and exclude worthless combination in early screening stages. This potentially caused to miss some useful interactions that would have appeared only in certain conditions, but allowed to complete model selection in time and without much confusion.

Discussion: Findings

Visual proofs available as graphs in the Appendix

Capacity Overfitting: The *Shallow network* achieved **lower test error** than *deeper models*, while having an **higher train error**, confirming that complex architectures easily memorize the training set on 500 samples.

Nested CV Role: Validation folds could not clearly define a best model. The generalization performance gap only emerged when comparing the results of the outer folds. A simple holdout validation would have chosen the final model almost at random between the best architectures.

Early Stopping Divergence: In the final retrain, *Error-Level* rule stopped prematurely (epoch ~ 3000) or not at all, while *Patience* reliably converged and yielded better results (epoch ~ 6500). This happens when using a fixed error level estimated from training on a smaller split of an already small dataset.

Decoupled Weight Decay: Since λ is totally independent of *learning rate*, when *Linear Decay* drives the η toward zero, on long runs the weights collapse because the regularization factor shrinks them without anything to counterbalance. However, *early stopping* safely avoids the network to train for that amount of time (started happening at epoch ~ 15.000 , *patience* stopped at epoch ~ 6000).

Conclusions

I agree to the disclosure and publication of my name, and of the results with preliminary and final ranking.

Implementing the framework in *C++* was a good way to approach machine learning programming from a lower level point of view, and an introduction to the actual foundations of higher level frameworks such as *TensorFlow*.

Designing the codebase with *OOP* in mind helped building a modular and expandable system from the start, without the need of rewriting from scratch at each new implemented feature.

Implementing a file-based dynamic *grid search* was fundamental to test out combinations, annotate details and experiment again fast.

However, this also showed the burden of *computing time*, so it is important to proceed in a schematic way to avoid being overwhelmed by results.

Blind Test Results: gwynn7_ML-CUP25-TS.csv ~
Gwynn7

Bibliography

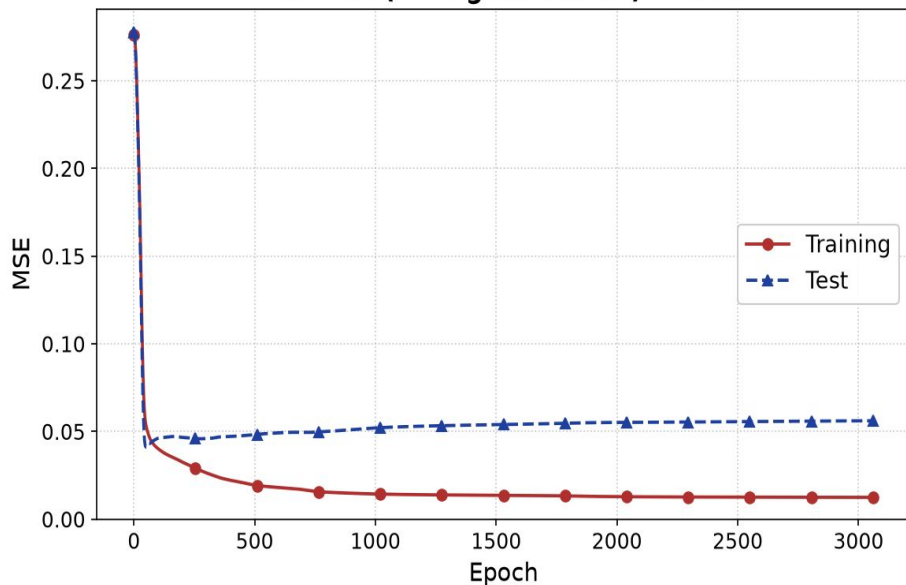
- [1] D. P. Kingma, J. Ba. Adam: A Method for Stochastic Optimization. 3rd International Conference on Learning Representations (ICLR), San Diego, 2015. arXiv:1412.6980. <https://arxiv.org/abs/1412.6980>
- [2] I. Loshchilov, F. Hutter. Decoupled Weight Decay Regularization. 7th International Conference on Learning Representations (ICLR), New Orleans, 2019. arXiv:1711.05101. <https://arxiv.org/abs/1711.05101>
- [3] X. Glorot, Y. Bengio. Understanding the Difficulty of Training Deep Feedforward Neural Networks. Proceedings of the 13th International Conference on Artificial Intelligence and Statistics (AISTATS), PMLR 9, pp. 249–256, 2010.
- [4] K. He, X. Zhang, S. Ren, J. Sun. Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification. IEEE International Conference on Computer Vision (ICCV), pp. 1026–1034, 2015. arXiv:1502.01852. <https://arxiv.org/abs/1502.01852>
- [5] V. Nair, G. E. Hinton. Rectified Linear Units Improve Restricted Boltzmann Machines. Proceedings of the 27th International Conference on Machine Learning (ICML), pp. 807–814, 2010.
- [6] CLI11: A Command Line Parser for C++11 and Beyond. <https://cliutils.github.io/CLI11/>
- [7] G. Guennebaud, B. Jacob and others. Header-only C++ template library for linear algebra. <https://libeigen.gitlab.io>
- [8] L. Dagum, R. Menon. OpenMP: An Industry Standard API for Shared-Memory Programming. IEEE Computational Science and Engineering, vol. 5, no. 1, pp. 46–55, 1998. doi:10.1109/99.660313
- [9] Z. Xianyi, M. Kroeker and others. OpenBLAS: An Optimized BLAS Library, based on GotoBLAS2 1.13 BSD. <https://www.openmathlib.org/OpenBLAS/>
- [10] J. Wnek. "MONK's Problems," UCI Machine Learning Repository, 1993. [Online]. Available: <https://doi.org/10.24432/C5R30R>.
- [11] L. Deng. The MNIST Database of Handwritten Digit Images for Machine Learning Research [Best of the Web]. IEEE Signal Processing Magazine, vol. 29, no. 6, pp. 141–142, 2012. doi:10.1109/MSP.2012.2211477

Appendix

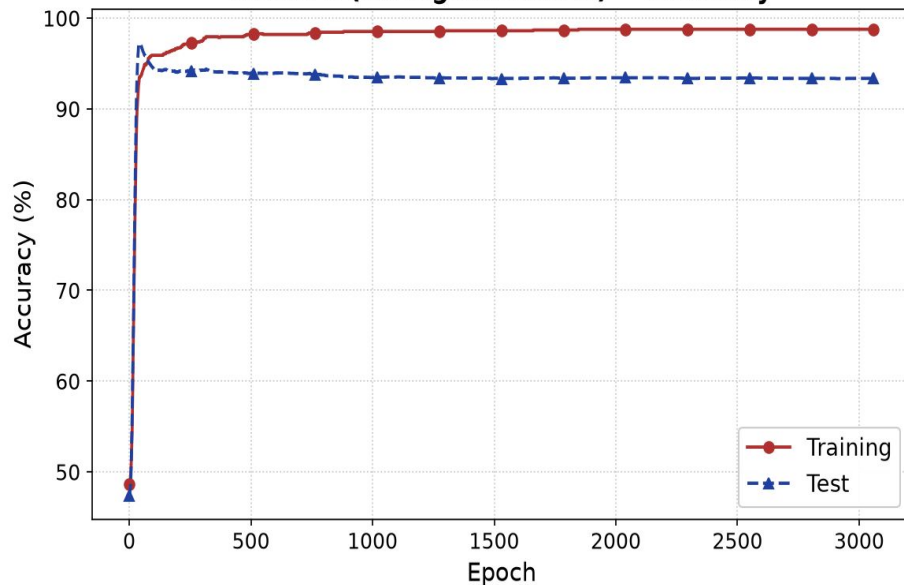
Additional graphs, plots and results

Monk 3 Results: Plot – No Regularization

MONK 3 (no regularization) – MSE

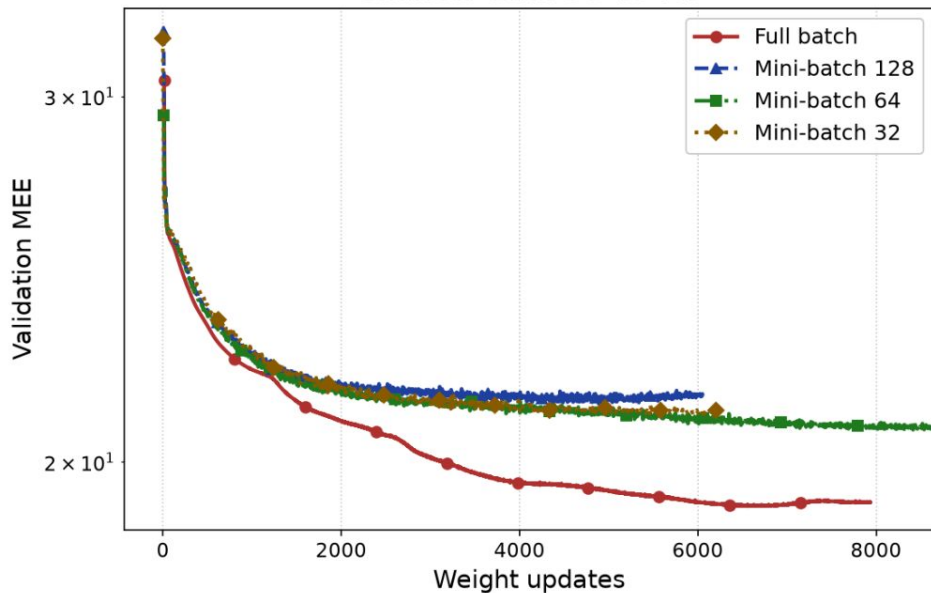


MONK 3 (no regularization) – Accuracy

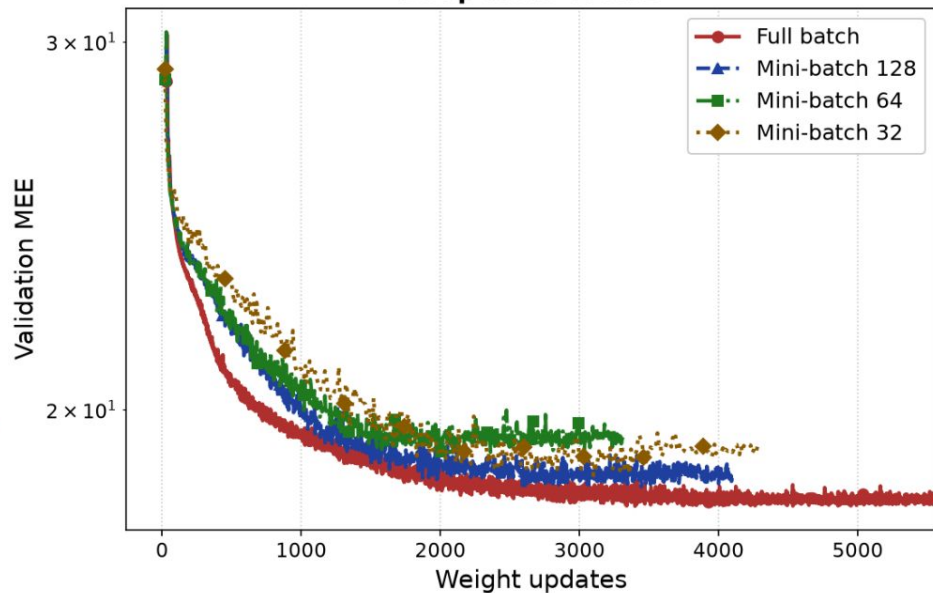


Hidden: Tanh – Output: Sigmoid – Init: Glorot – Loss: MSE – Optimizer: SGD – Normalization: None

Shallow Tanh 96-48-24

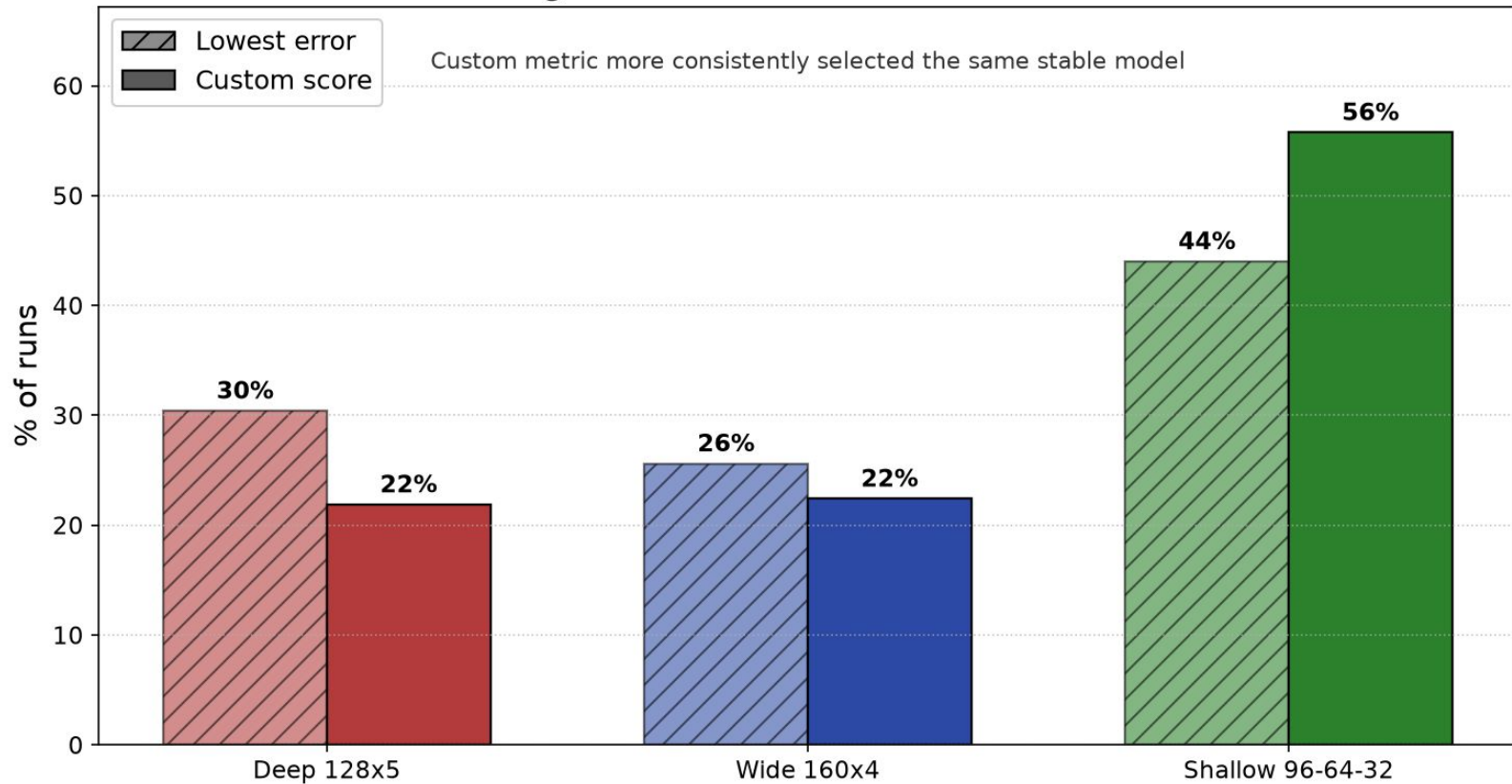


Deep ReLU 128x5



[ML-CUP] Loss Curve stability of models trained at different batch sizes (and different learning rate)

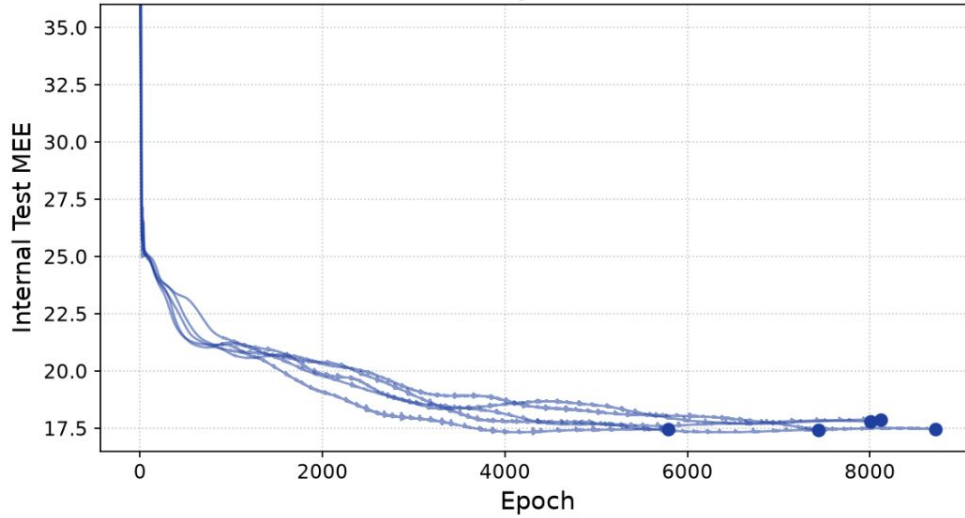
Single Run Model Selection Criteria



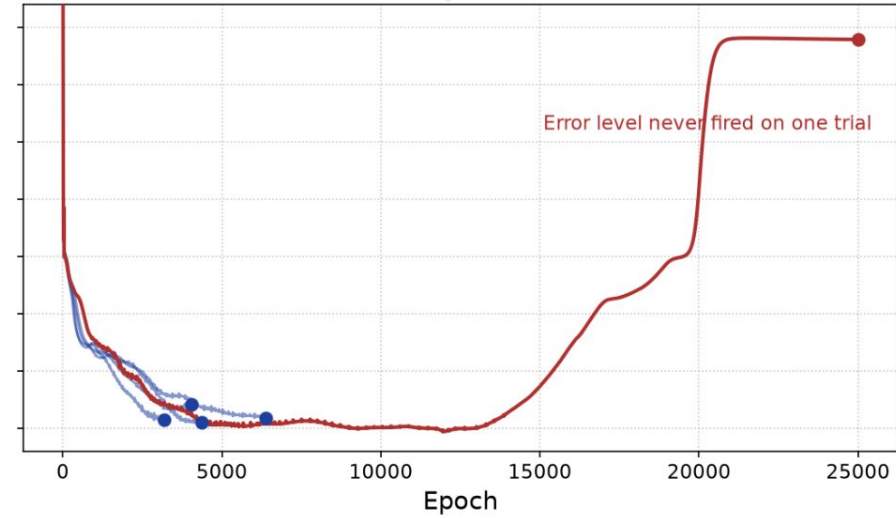
[ML-CUP] How often, each run, the criteria picked each model, showing the consistency of the custom metric

Stopping method on every trial of the Final Retrain

Patience (stops at 5790-8715)

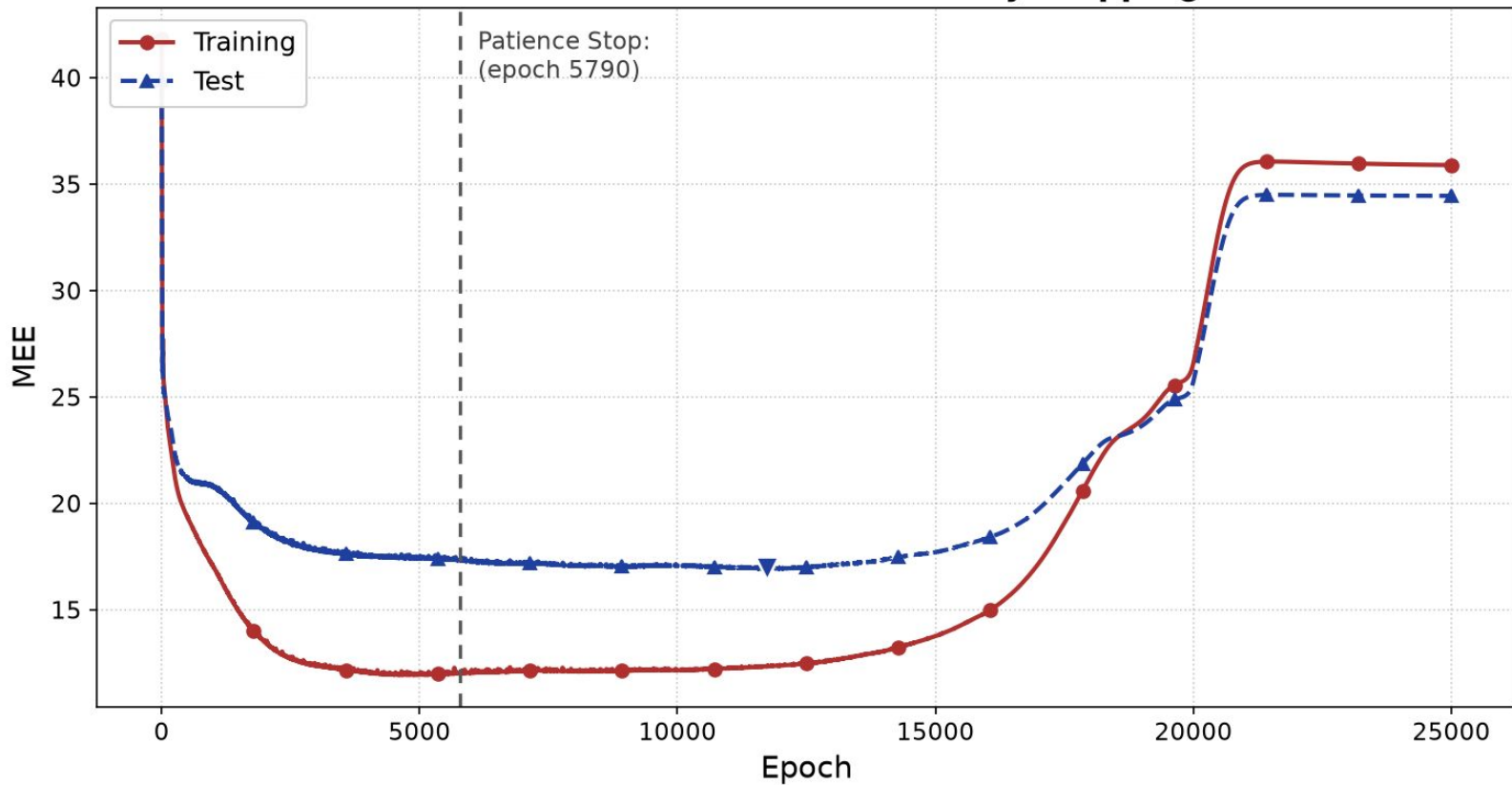


Error level (stops at 3173-24999)



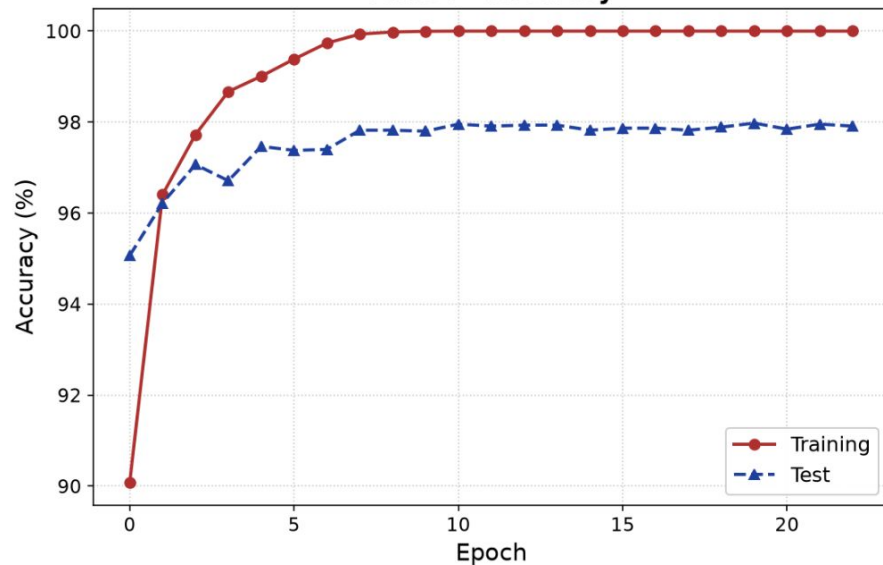
[ML-CUP] Epochs at which Early Stopping rules interrupt training, showing the reliability of Patience

Final Model Retrain without Early Stopping

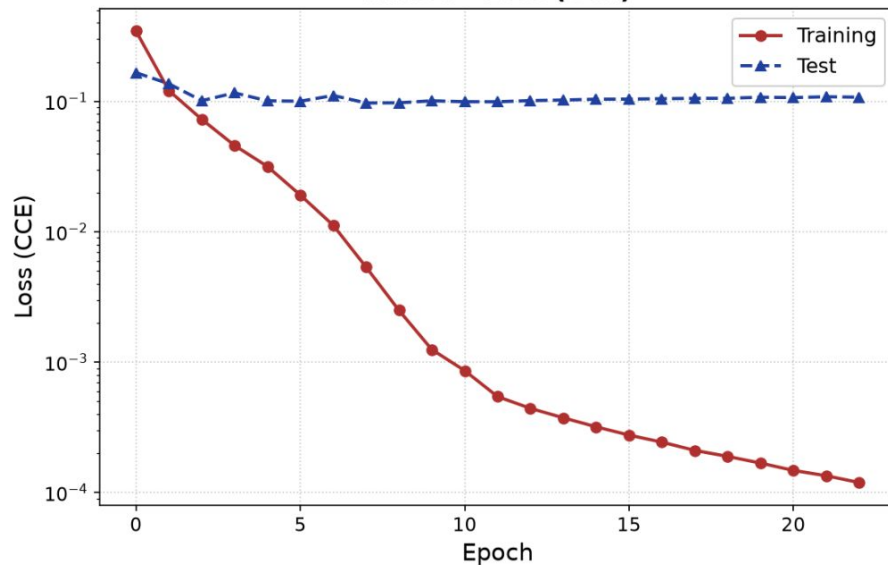


[ML-CUP] Complete training at max epochs without early stopping, showing the final shrinking of the weights

MNIST - Accuracy



MNIST - Loss (CCE)



Network	Hidden	Output	Init	Optim	Loss	Norm	Batch	η	λ	Train Acc	Test Acc
512-256	ReLU	Softmax	He	Adam	CCE	MinMax	128	0.001	0.001	100.00%	97.38%

Best result of a very quick grid search on MNIST dataset